

QueryMind

AI SQL ASSISTANT

Built on n8n | Ollama | MySQL | PostgreSQL

Project Build Report — Problems Faced — Lessons Learned

Personal Project | July 2026

1. Executive Summary

The AI SQL Assistant is an autonomous natural language database query system built inside n8n. It allows any user to type a plain-English question and receive an accurate, conversational answer sourced directly from a live MySQL database — without writing a single line of SQL.

Attribute	Detail
Project Name	AI SQL Assistant (sql_helper)
Platform	n8n — self-hosted via Docker
LLM	Ollama llama3.1:8b (local) Claude Haiku / GPT-4o recommended for production
Database	MySQL 8.0.46 — accessed via host.docker.internal
Memory	PostgreSQL Chat Memory — long-term, persistent across sessions
Query Logging	MySQL query_logs table — full audit trail of every Q&A
Status	Functional — basic to intermediate SQL queries working end-to-end

2. Project Goal

2.1 Original Objective

The initial goal was to build a fully general-purpose AI SQL assistant with the following capabilities:

- **1.** Accept natural language questions from any user via a built-in chat interface.
- **2.** Autonomously discover database schema at runtime without any hardcoding.
- **3.** Generate accurate SQL queries and execute them against a live MySQL database.
- **4.** Return results as clean, plain-English conversational answers.
- **5.** Persist conversation history across sessions using long-term memory.

2.2 What Was Actually Achieved

Open-source local models running via Ollama proved insufficient for fully autonomous operation. The following reflects what the final implementation delivers:

- **1.** Chat interface works — users can type plain-English questions and receive answers.
- **2.** Schema is hardcoded in the system prompt — the model cannot reliably discover it at runtime without hallucinating. Currently configured for the accident database specifically.
- **3.** SQL generation and execution works — basic SELECT, filtered queries, and single-table aggregations run reliably.
- **4.** Long-term memory works — PostgreSQL Chat Memory persists conversation history across sessions successfully.

2.3 With Cloud LLMs

All original objectives become fully achievable by replacing the Ollama node with a cloud LLM. Cloud models such as Claude Haiku or GPT-4o handle runtime schema discovery, multi-database queries, complex JOINS, and subqueries without extra prompt engineering or hardcoding. The workflow architecture already supports this — only the LLM node needs to be swapped.

3. Final Workflow Architecture

3.1 Node Pipeline

#	Node	Type	Role
1	When chat message received	Chat Trigger	Receives user question via n8n built-in Chat UI
2	AI Agent	Tools Agent v3.1	Orchestrates LLM and tool loop — generates SQL, interprets results
3	Ollama Chat Model	LLM — llama3.1:8b	Language model powering reasoning and SQL generation
4	run_sql	MySQL Tool	Executes AI-generated SQL against MySQL and returns raw results
5	Chat Memory	PostgreSQL Memory	Persists full conversation history across sessions permanently
6	sql_log	MySQL Insert	Logs every question and answer to a query_logs audit table
7	Respond to Webhook	Webhook Response	Returns the final answer back to the Chat UI

3.2 Final System Prompt

```
You are a MySQL assistant with one tool: run_sql.
ALWAYS call run_sql for every response. No exceptions.

DATABASE: accident
SCHEMA:
Table: accident      - report_no VARCHAR (PK), acc_date DATE, location VARCHAR
Table: car           - license VARCHAR (PK), model VARCHAR, caryear YEAR
Table: owns          - driver_id VARCHAR, dri_name VARCHAR, license VARCHAR (FK)
Table: participated  - driver_id VARCHAR, license VARCHAR, report_no VARCHAR,
                      damage_amount INT

RELATIONSHIPS:
- participated links accident + car + driver
- owns links driver to their car via license
- Use participated for damage amounts
- Use owns for driver names
- Join owns and participated on driver_id + license

STEPS (each as a separate tool call):
  1. run_sql('USE accident;')
```

```
2. run_sql('SELECT ... LIMIT 1000;')
```

RULES:

- 1. Only SELECT. Never INSERT/UPDATE/DELETE/DROP.
- 2. Only use column names from schema above. Never guess.
- 3. Never return raw JSON or SQL to the user.
- 4. Summarise in plain English. Lists as bullet points.
- 5. NEVER use LIMIT inside a subquery.
- 6. Always prefix columns with table alias in JOINS.
- 7. Never put semicolon before LIMIT.
- 8. If 0 rows returned, say so clearly.

4. How It Was Built – Phase by Phase

Phase	What Was Done	Outcome
1	Set up Chat Trigger node with public URL, custom greeting and placeholder text	Chat UI live and accessible
2	Added AI Agent node (Tools Agent v3.1) with system prompt and 10 max iterations	Agent framework configured
3	Connected Ollama Chat Model node using host.docker.internal:11434	Local LLM connected to agent
4	Added MySQL Tool node (run_sql) with \$fromAI() expression for dynamic query injection	Database tool wired to agent
5	Fixed MySQL credentials — created n8n@% user with SELECT ON *.* grant	MySQL connectivity established
6	Added Simple Memory, then upgraded to PostgreSQL Chat Memory for persistence	Long-term memory working
7	Added sql_log node to INSERT every Q&A into a query_logs audit table in MySQL	Full audit trail in place
8	Hardcoded database schema into system prompt for accurate SQL generation	JOIN and aggregation accuracy improved

5. Problems Faced and How They Were Fixed

5.1 Infrastructure and Connectivity

Problem	Root Cause	Fix Applied
MySQL IP breaks on WiFi reconnect	Mac WiFi assigns a new DHCP IP on each network reconnect	Created n8n@% user (wildcard host) and used host.docker.internal permanently

5.2 MySQL Credential Issues

Problem	Root Cause	Fix Applied
INSERT denied on query_logs table	n8n user only had SELECT — no INSERT permission	Granted INSERT on query_logs table specifically to n8n user

5.3 LLM and Model Issues

Problem	Root Cause	Fix Applied
llama3 does not support tools	Base llama3 has no function/tool-calling capability built in	Switched to llama3.2, then mistral, then llama3.1:8b
sqlcoder does not support tools	SQLCoder is a pure SQL model with no tool-calling support	Abandoned SQLCoder — incompatible with n8n agent architecture
qwen2.5:7b connection timeout	Model too large for MacBook Air available RAM — caused timeout on inference	Switched to qwen2.5:3b and later llama3.1:8b
Model explains steps instead of executing	Mistral verbose by default — ignores silent execution instructions	Added explicit rules: never explain, never show SQL, execute silently

5.4 SQL Generation Issues

Problem	Root Cause	Fix
Wrong table names hallucinated	Model guessed column names instead of reading SHOW TABLES result (weak model)	Hardcoded full schema in system prompt — no runtime discovery

6. LLM Model Comparison

Multiple models were tested during development. The table below documents observed performance across key SQL capabilities on a MacBook Air (Apple Silicon).

Model	Tool Calling	Basic SELECT	JOINS	Aggregations	Speed	Verdict
llama3 (base)	No	Yes	N/A	N/A	Fast	Unusable — no tool support
llama3.2	Yes	Yes	Weak	Weak	Fast	Too weak for complex SQL
mistral	Unreliable	Yes	Weak	Weak	Fast	Verbose, unreliable tool calls
qwen2.5:7b	Yes	Yes	Moderate	Moderate	Times out	Too large for MacBook Air
sqlcoder	No	Yes	Good	Good	Slow	Unusable — no tool support
llama3.1:8b	Yes	Yes	Moderate	Moderate	Slow	Current — works for basic queries
Claude Haiku (cloud)	Yes	Yes	Excellent	Excellent	Very fast	Recommended for production
GPT-4o Mini (cloud)	Yes	Yes	Excellent	Excellent	Very fast	Recommended for production

7. Recommendation for Production Use

Note: Local models running via Ollama are suitable for development, learning, and demos only. For production use involving complex JOIN queries, aggregations, and large schemas, a cloud-based LLM is strongly recommended.

7.1 Why Cloud LLM for Higher-Level Usage

- Reliable tool calling — never fails to invoke `run_sql`.
- Handles complex JOINS, subqueries, and aggregations without extra prompt engineering.
- No hallucination of table or column names when schema is provided.
- Sub-second response time regardless of query complexity.
- Works with schemas of 1 lakh+ rows and 10+ tables without degradation.
- No local hardware constraints — runs entirely on the provider's infrastructure.

7.2 How to Switch from Ollama to Claude in n8n

Step	Action
1	Go to console.anthropic.com and create an API key under API Keys.
2	Delete the Ollama Chat Model node from the n8n canvas.
3	Click the + on the Chat Model slot of the AI Agent node.
4	Search for Anthropic Chat Model and select it.
5	Create a new credential and paste the Anthropic API key.
6	Set model to claude-haiku-4-5 (fast and low cost) or claude-sonnet-4-6 (highest capability).
7	Save and test — all SQL generation issues resolve immediately.

7.3 Estimated Cost — Claude Haiku

Usage Level	Queries per Day	Estimated Monthly Cost
Development and Testing	50 queries	Approx. Rs. 30 per month
Small Team (5 users)	200 queries	Approx. Rs. 120 per month
Production (50 users)	2000 queries	Approx. Rs. 1,200 per month

8. Key Lessons Learned

- **1.** Always use host.docker.internal when n8n runs in Docker. localhost and 127.0.0.1 point to the container itself, not the host machine. Every local service — MySQL, Ollama, PostgreSQL — must be reached via host.docker.internal. This applies to any tool running inside Docker trying to reach something on the host.
- **2.** Test multiple models before concluding the architecture is broken. Six models were tested during this build. Each failed differently and each failure revealed something specific about what the model could and could not do. Model evaluation is a core skill in AI engineering — do not stop at the first model that partially works.
- **3.** Open-source models handle basic SELECT queries but sometimes fail at multi-table JOINS, aggregations, and subqueries without heavy prompt engineering. Cloud models like Claude or GPT-4o resolve these issues.
- **4.** Long system prompts hurt small models. A 400-word prompt that performed poorly became more effective when trimmed to under 150 words. Put the most critical instructions at the very top.
- **5.** Change one thing at a time and use the Logs tab, not just the chat. The n8n Logs tab shows exactly which nodes fired, in what order, and what data passed through — debugging from chat output alone is guessing.

9. Final Configuration Reference

9.1 MySQL User Setup

Run these commands inside MySQL Workbench or the MySQL terminal to create a dedicated user for n8n.

Step 1 — Create the n8n user

```
-- % means the user can connect from any IP address
CREATE USER 'n8n'@ '%' IDENTIFIED BY 'yourpassword';
```

Step 2 — Grant read-only access across all databases

```
-- SELECT only — prevents AI from modifying any data
GRANT SELECT ON *.* TO 'n8n'@ '%';
```

Step 3 — Grant INSERT on the audit log table only

```
-- Required only if you use the sql_log node for query logging
GRANT INSERT ON your_database.query_logs TO 'n8n'@ '%';
```

Step 4 — Apply changes

```
FLUSH PRIVILEGES;
```

Step 5 — Create the audit log table

```
CREATE TABLE query_logs (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  question    TEXT,
  answer      TEXT,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

n8n Credential Settings

Field	Value
Host	host.docker.internal
Port	3306
Database	leave blank (to allow all databases)

Field	Value
User	n8n
Password	yourpassword
SSL	Disabled

9.2 PostgreSQL Memory Setup

```
# Install PostgreSQL via Homebrew
brew install postgresql@16
brew services start postgresql@16

# Create the memory database
psql postgres
CREATE DATABASE n8n_sql_memory;
\q

# n8n Postgres credential settings:
# Host:      host.docker.internal
# Port:      5432
# Database:  n8n_sql_memory
# User:      <your_mac_username>
# Password:  leave blank
```